Especificação do Software e do Sistema

Cadeia de Frio IoT com Assinatura ECDSA e Ledger de Hash, MNR 2026

LACOR, Universidade Tiradentes (UNIT), Aracaju

Versão 1.0, 19 de outubro de 2026

Sumário

Seções	Seções (continuação)
0. Leia isto primeiro (resumo para quem vai programar)	11. Camada 3: extras
1. Contexto do projeto	12. Plano de testes
2. O problema e a ideia	13. Artigo versus demonstração
3. Escopo, camadas e critérios de aceite	14. Roteiro da demonstração e material
4. Arquitetura	15. Plano de implementação (para o Claude Code)
5. Hardware e ligações	16. Cronograma
6. Especificação do pacote (contrato central)	17. Riscos e plano B
7. Criptografia e modelo de ameaças	18. Stack consolidada
8. Firmware do nó sensor	19. Glossário
9. Firmware do receptor e ponte serial	20. Referências e fontes de consulta
10. Gateway, ledger e tela de demonstração	

0. Leia isto primeiro (resumo para quem vai programar)

Este documento é a fonte única de contexto para o desenvolvimento do software do projeto MNR 2026 da LACOR. Ele foi escrito para ser anexado a uma sessão de Claude Code e para ser lido por qualquer pessoa da equipe. Se você só ler uma página, leia esta.

0.1 O que o sistema é, em palavras simples

É um sistema que **prova que a temperatura registrada durante o transporte de vacinas ou órgãos não foi alterada por ninguém depois de medida**.

Funciona em três passos:

1. Um **dispositivo pequeno dentro da caixa térmica** (o "nó sensor") mede a temperatura a cada poucos segundos. Se estiver fora da faixa segura (2 a 8 °C), ele acende um alerta no próprio dispositivo.
2. A cada medida, o nó monta um pacote de 88 bytes e o **assina digitalmente** (ECDSA, curva P-256). A assinatura é como um lacre matemático: se qualquer byte do pacote mudar depois, a assinatura deixa de bater. O nó envia o pacote por rádio **LoRa**, que alcança vários quilômetros com pouca energia.
3. Um **receptor (gateway)** recebe o pacote, **confere a assinatura**, descarta o que for falso, repetido ou adulterado, e grava o que é válido num **registro encadeado por hash** (cada registro carrega a impressão digital do anterior, como uma corrente). Se alguém editar um registro antigo no banco de dados, a corrente quebra e o sistema aponta exatamente onde.

O resultado prático: quando a carga chegar ao destino, é possível mostrar um histórico de temperatura **confiável e auditável**, e demonstrar ao vivo que uma tentativa de fraude é detectada.

0.2 O que NÃO é

- Não é um app de celular, nem um site comercial. A "interface" é uma tela local simples para a demonstração.
- Não é uma blockchain pública. O "ledger distribuído" do título do artigo é, na implementação, uma **cadeia de hash replicada**. Não use a palavra "blockchain" em código, documentação ou na fala da apresentação.
- Não é LoRaWAN completo. Usaremos **LoRa ponto a ponto** entre duas placas, emulando o papel de um sensor e de um gateway. Não há concentrador LoRaWAN nem servidor de rede.
- Não depende de nuvem para funcionar. Nuvem (AWS) e GPS são extras.

0.3 O que precisa existir no fim (definição de pronto)

A demonstração no Nacional (23 a 29 de novembro de 2026, João Pessoa) precisa mostrar, ao vivo e sem intervenção manual além de apertar botões:

- **A.** O nó lendo a temperatura real do sensor e mostrando no OLED, com alerta visual (LED) acima de 8 °C ou abaixo de 2 °C.
- **B.** O pacote assinado chegando ao receptor por LoRa, e o gateway marcando "ACEITO".
- **C.** Um pacote adulterado de propósito (mudar a temperatura de 8,6 para 5,0) sendo marcado "ASSINATURA INVÁLIDA".

- **D.** Um pacote reenviado (replay) sendo marcado “REPLAY”.
- **E.** O registro encadeado sendo verificado como íntegro, e depois uma edição manual no banco sendo detectada pela corrente.
- **F.** Medições reais para a tabela do artigo: tempo de assinatura no ESP32, tempo no ar do pacote, taxa de entrega.

Tudo isso existe hoje, **exceto o hardware real**, em simulação Python (item 1.6). O trabalho restante é: firmware das duas placas, ponte serial no Raspberry Pi, tela da demonstração, medições, e (se sobrar tempo) extras.

0.4 Ordem de trabalho resumida

1. Já feito: núcleo em Python (pacote, ECDSA P-256, decisão, ledger, gateway, simulador) com 16 testes passando.
2. Fazer agora, sem hardware: vetores de teste, ponte serial simulada, tela do gateway, esboço do firmware.
3. Fazer quando as placas chegarem: firmware do nó, firmware do receptor, integração, medições.
4. Fazer por último: nuvem, GPS, acabamento, ensaio, vídeo.

A ordem detalhada, tarefa por tarefa, com critério de aceite, está na Seção 15.

1. Contexto do projeto

1.1 O evento

A **MNR (Mostra Nacional de Robótica)** é uma competição da **RoboCup Brasil**, o maior evento de robótica da América Latina. A fase nacional de 2026 acontece de **23 a 29 de novembro de 2026, em João Pessoa (PB)**. O trabalho da LACOR foi **aprovado nos Anais** e classificado para o Nacional, na modalidade de Ensino Superior, Pós-graduação e Pesquisa Tecnológica, que avalia um artigo científico. Há ainda vídeo de 3 a 5 minutos e banner, que já foram produzidos.

1.2 A equipe

Pessoa	Papel
Vinicius Prado Sobral	Presidente da LACOR, liderança técnica do desenvolvimento
Matheus Felipe Pereira Santos	Equipe
Júlia Lira Santana Rodrigues da Cunha	Equipe
Manuela Silva Santos Souza	Equipe
Iasmin Silva de Menezes	Equipe
Prof. Luiz Gomes da Cunha Neto	Orientador

A **LACOR** (Liga Acadêmica de Computação e Inovação em Robótica) é a liga da UNIT que desenvolve o projeto.

1.3 O artigo

Título: *Logística Segura de Cadeia de Frio para Órgãos e Vacinas: Arquitetura IoT e Ledger Distribuído com Validação Criptográfica Computacional para o Contexto Brasileiro e Nordestino.*

O artigo aprovado (versão final dos Anais) tem **5 páginas**, 3 figuras, 3 tabelas e 7 referências, no modelo oficial da MNR. Ele propõe uma arquitetura em **quatro camadas**:

1. **Hardware de borda:** ESP32-WROOM-32 com sensor SHT31-D, GPS NEO-6M, acelerômetro MPU-6050, relógio DS3231, Reed Switch, módulo LoRa SX1276, bateria de 3,7 V e 5.000 mAh (autonomia especificada superior a 72 h, custo de US$ 26 a US$ 85 por unidade).
2. **Protocolo criptográfico:** a cada ciclo de **60 s**, o firmware lê os sensores, monta um **payload em JSON**, calcula SHA-256, assina com ECDSA (biblioteca **micro-ecc**) e monta um **bloco encadeado** (payload, hash, assinatura e hash do bloco anterior).
3. **Conectividade redundante:** LoRaWAN (primário), GSM/4G com MQTT sobre TLS (secundário) e armazenamento local em LittleFS com envio em lote (terciário, "store-and-forward").
4. **Nuvem e painel:** AWS IoT Core valida assinatura, continuidade da cadeia e conformidade térmica, e dispara alerta para o painel.

O foco geográfico é o Nordeste brasileiro (Semiárido, energia solar, rota piloto Fortaleza a Juazeiro do Norte).

O que o artigo afirma ter feito, e o que declara como não feito. O artigo é transparente: ele apresenta uma **simulação em Python** (biblioteca `ecdsa`, curva SECP256k1, semente 42) e, na Tabela 1, marca todo o hardware (ESP32, sensores, LoRa, micro-ecc) como **"Planejado"**. A conclusão diz expressamente que a validação em hardware físico "ainda não foi executada". Resultados da simulação: classificação térmica correta em 14 de 14 casos (2 a 15 °C, limiar 8 °C), detecção de adulteração em 50 de 50 casos, e latência média de 0,59 ms (n = 100, mínimo 0,51, máximo 0,69) em CPU de PC. A Tabela 3 lista nove hipóteses simplificadoras e a evolução proposta para cada uma. **Consequência prática:** nada do que a equipe mostrar em hardware contradiz o artigo, e cada item que o protótipo concretizar é uma evolução que o artigo já anunciou.

Pontos em que o artigo e a implementação diferem e que precisam ser declarados com honestidade:

- **Curva.** O artigo usou **secp256k1**. O código atual usa **P-256 (secp256r1)**. A micro-ecc suporta as duas, então trocar para secp256k1 e eliminar essa diferença é possível (ver decisão pendente na Seção 7.4). Se ficar em P-256, diga: "a simulação usou secp256k1 e o protótipo usa P-256, ambas de 256 bits".
- **Latência.** Os 0,59 ms foram medidos com biblioteca Python pura no PC. O script `tools/repro_artigo.py` reproduz os três testes com o código do projeto (a latência sai bem menor porque a biblioteca de hoje usa OpenSSL, o que **não** é comparável). A medida que importa é a do **ESP32**, que deve ser feita de novo.
- **Intervalo de envio.** O artigo especifica ciclo de **60 s**. O padrão de bancada do projeto é 10 s, para a demonstração fluir. Em `config.h` o valor de referência do artigo (60 s) deve existir e a demonstração deve declarar o intervalo usado.
- **Formato do dado.** O artigo descreve payload em **JSON**. Um JSON típico passa de 150 bytes e, com assinatura, aproxima-se do limite de 255 bytes do LoRa. O projeto usa pacote **binário de 88 bytes**. É uma escolha técnica a declarar, com o argumento de tempo no ar menor.

- **Onde fica a corrente de hashes.** O artigo monta o bloco encadeado **no nó**. O projeto monta a corrente **no gateway** (ledger SQLite). Opcional da Camada 3: o nó também guarda um registro local em LittleFS (store-and-forward).
- **Rede.** O artigo menciona LoRaWAN, GSM/4G e MQTT. O protótipo usa **LoRa ponto a ponto**, sem GSM e sem concentrador LoRaWAN.
- **Placa e rádio.** O artigo planeja **ESP32-WROOM-32 com SX1276 (TTGO LoRa32)**. A compra foi a **Heltec WiFi LoRa 32 V3 (ESP32-S3 com SX1262)**. Declarar a troca e o motivo (disponibilidade e custo).
- **Sensores não comprados.** MPU-6050, DS3231, Reed Switch e bateria de 5.000 mAh constam do artigo e não estão na compra. Sem o DS3231, o carimbo de tempo vem do gateway ou do relógio interno (declarar). A autonomia de 72 h é **especificação**, nunca dizer que foi medida.
- **Frequência.** O artigo não cita frequência. O protótipo opera em **915 MHz**, faixa do Brasil, sem divergência a declarar.

1.4 O que as regras oficiais da MNR permitem (o que foi lido)

Foram lidas páginas e manuais oficiais da MNR (site mnr.robocup.org.br e manual de 2025). Conclusões, com a ressalva de que a ficha de avaliação e o Manual do Expositor de 2026 **não puderam ser lidos**:

- O projeto **não precisa estar concluído**. O artigo e o vídeo devem cobrir desenvolvimento, próximos passos e objetivo final, com “os resultados obtidos até o momento”.
- O artigo deve descrever “o protótipo, as partes e elementos principais, além de evidenciar a programação”.
- Na Mostra presencial, “espera-se que você possa realizar uma demonstração física de seu trabalho”.
- Cada trabalho é avaliado por três avaliadores. Os critérios listados são adequação à temática, qualidade do texto, qualidade do material multimídia, potencial de difusão de ciência e tecnologia e potencial de evolução do trabalho.
- **Não há regra publicada que obrigue o uso de peças específicas.** O que pesa é a coerência entre o que o artigo diz e o que se mostra. Troca de peça equivalente é aceitável se for **declarada**.
- A **ausência** da equipe no dia e horário marcados leva à desclassificação.
- Contato oficial para dúvidas: mnr@robocup.org.br.

Consequência para o software: tudo que a demonstração mostrar deve estar descrito no artigo ou declarado como diferença. Existe uma tabela “Artigo × demonstração” na Seção 13 para isso.

1.5 Hardware e situação de compra

O pedido no AliExpress foi feito em 19/10/2026 e todos os itens saem da China. O despacho pode levar até 9 dias, a meta de entrega é 10/11/2026 e o reembolso por atraso só vem em 50 dias, o que passa da competição. Por isso:

- **Checagem em 21/10/2026.** Se o pedido ainda estiver “Processando”, cancelar e comprar a Heltec V3 em loja nacional (Usinainfo, Smartkits ou Mercado Livre).
- **Data de corte em 5/11/2026.** Se até lá o hardware não estiver funcionando, apresentar o plano B: a simulação, o gateway e o vídeo, com a tabela “Artigo × demonstração”.

Lista de peças:

Peça	Quantidade	Função
Heltec WiFi LoRa 32 V3 (ESP32-S3, rádio SX1262, OLED 0,96”)	2	Uma é o nó sensor, a outra é o receptor do gateway
Sensor SHT31-D (I2C)	2	Temperatura e umidade. O segundo é reserva
Módulo GPS NEO-6M (UART)	1	Opcional. Posição do nó
Raspberry Pi 3B (já do LACOR)	1 a 2	Roda o gateway em Python, ledger e tela
Arduino Uno, protoboard, jumpers, LEDs, resistores (já do LACOR)	vários	Montagem e testes
Power bank USB, cabos USB-C de **dados**	2	Alimentação e programação

Se algum item for substituído (por exemplo, um sensor DS18B20 no lugar do SHT31-D), a troca deve ser declarada, e a camada de abstração do sensor (Seção 8.4) existe para isso.

1.6 Estado atual do código

Repositório `mnr-coldchain`, Python 3.10 ou superior, única dependência `cryptography`. Já implementado e testado, sem hardware:

Arquivo	O que faz
`src/coldchain/packet.py`	Formato binário do pacote (24 bytes de dados mais 64 de assinatura)
`src/coldchain/crypto.py`	ECDSA P-256 e SHA-256, assinatura bruta r e s de 64 bytes, exportação de chave pública
`src/coldchain/decision.py`	Classificação térmica: OK, ALERTA, ALARME
`src/coldchain/ledger.py`	Registro SQLite encadeado por hash e verificação da corrente
`src/coldchain/gateway.py`	Valida tamanho, versão, nó conhecido, assinatura e replay, depois grava
`src/coldchain/node_sim.py`	Simula o nó sensor: mede, classifica, assina
`run_demo.py`	Demonstração ponta a ponta sem hardware
`tests/test_coldchain.py`	16 testes unitários (todos passam)
`docs/PACOTE.md`	Especificação do formato do pacote

O que **falta**: firmware, ponte serial, tela, vetores de teste, medições, extras. A Seção 15 lista cada um.

2. O problema e a ideia

2.1 O problema

Vacinas e órgãos para transplante precisam ficar em uma faixa estreita de temperatura (tipicamente 2 a 8 °C para vacinas). Se a cadeia de frio quebra, o produto pode perder eficácia ou ficar inutilizável, e muitas vezes isso **não é visível a olho nu**. No Brasil, e especialmente no Nordeste, há trajetos longos, estradas com cobertura de rede irregular e regiões semiáridas com calor intenso.

Os registradores de temperatura comuns têm duas fraquezas:

- **Cobertura de rede.** Muitos dependem de celular. Onde não há sinal, o dado se perde ou chega atrasado.
- **Confiança no registro.** Um registro em planilha ou em banco comum pode ser editado depois, por engano ou de má-fé, para esconder uma falha (“limpar” o histórico antes de uma auditoria ou de um seguro).

2.2 A ideia

Atacar os dois pontos com tecnologias de baixo custo:

- **LoRa** para alcance longo e baixo consumo, sem depender de celular.
- **Assinatura digital no próprio sensor**, de modo que o dado já nasce “lacrado”. Quem recebe pode provar que aquele número veio daquele dispositivo e não foi mudado.
- **Registro encadeado por hash**, de modo que apagar ou editar uma leitura antiga seja detectável.

2.3 O que a assinatura garante e o que ela não garante

Isso precisa estar claro no código, na apresentação e nas respostas aos avaliadores.

Garante:

- **Integridade:** o pacote não foi alterado depois de assinado.
- **Autenticidade:** o pacote foi assinado por quem tem a chave privada daquele nó.
- **Detecção de replay:** pacotes repetidos são descartados pelo contador `seq`.
- **Detecção de adulteração do histórico:** a corrente de hash acusa edição posterior.

Não garante:

- **Confidencialidade.** O pacote vai em claro no ar. Qualquer pessoa pode ouvir a temperatura, mas não pode forjar. Criptografar com AES-GCM é uma evolução possível, fora do escopo.
- **Que o sensor mediu certo.** Se o sensor for fisicamente enganado (aquecido de propósito, por exemplo), a assinatura será válida para um dado falso. A assinatura protege o **caminho**, não a **medição**. Isso é uma limitação honesta e deve ser dita se perguntarem.
- **Proteção da chave privada.** No protótipo, a chave fica no firmware. Em produção seria guardada num elemento seguro. Se alguém extrair a chave, pode forjar.
- **Disponibilidade.** Interferência de rádio (jamming) pode impedir a entrega. O sistema detecta ausência de pacotes, mas não impede o bloqueio.

3. Escopo, camadas e critérios de aceite

O escopo é dividido em camadas. Em caso de falta de tempo, **corta-se de trás para frente**.

3.1 Camada 1, obrigatória

Sem isso, não há demonstração.

- **C1.1** Nó lê o SHT31-D por I2C e mostra a temperatura no OLED.
- **C1.2** Nó classifica (OK, ALERTA, ALARME) e acende o LED do alarme.
- **C1.3** Nó monta o pacote de 88 bytes e assina com ECDSA P-256.
- **C1.4** Nó envia por LoRa em 915 MHz.
- **C1.5** Receptor (segunda Heltec) recebe e imprime o pacote pela USB.
- **C1.6** Gateway verifica a assinatura e rejeita pacote adulterado.

Aceite da camada 1: um pacote gerado pelo firmware passa na verificação do gateway Python **sem nenhuma alteração**, e um pacote com 1 bit alterado é rejeitado.

3.2 Camada 2, forte

- **C2.1** Ponte serial: o Raspberry Pi lê as linhas da Heltec receptora e chama o gateway.
- **C2.2** Ledger SQLite com cadeia de hash e verificação.
- **C2.3** Tela de demonstração (web local) com leituras, status, estado do ledger e botões de simulação de ataque.
- **C2.4** Anti-replay com persistência do contador no nó.

Aceite da camada 2: a demonstração A a E da Seção 0.3 roda do início ao fim em menos de 5 minutos.

3.3 Camada 3, extras

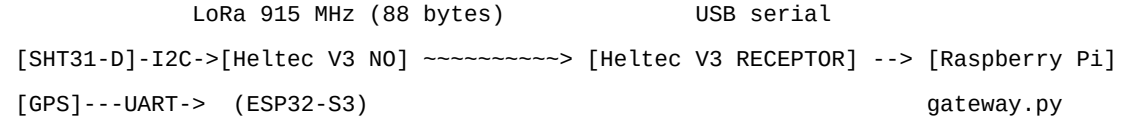
- **C3.1** MQTT local (Mosquitto) publicando cada leitura aceita.
- **C3.2** Integração com AWS IoT Core e DynamoDB.
- **C3.3** GPS NEO-6M no nó, com latitude e longitude no pacote.
- **C3.4** Alimentação com bateria e painel solar (apenas como discussão no artigo).

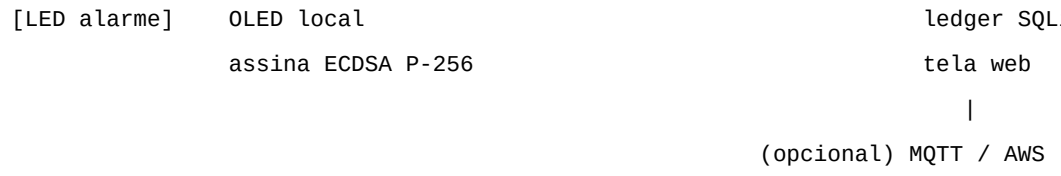
3.4 Fora do escopo (decisão explícita)

LoRaWAN real, módulo GSM, criptografia de payload, rotação de chaves, múltiplos nós em rede, aplicativo móvel, certificação Anatel, hardware de produção. Esses itens aparecem no artigo como trabalho futuro e **não** devem consumir tempo de desenvolvimento.

4. Arquitetura

4.1 Visão geral





Há **dois domínios de código** e **uma linha de contrato** entre eles:

- **Firmware (C++ no ESP32-S3):** duas imagens, uma para o nó, outra para o receptor. Ficam em `firmware/`.
- **Software de gateway (Python no Raspberry Pi):** `src/coldchain/`.
- **Contrato:** o formato do pacote (Seção 6) e o protocolo serial (Seção 9.3). Qualquer mudança nesses dois exige atualizar documentação, testes e os dois lados.

4.2 Componentes e responsabilidades

Componente	Roda em	Responsabilidade	Não deve fazer
Nó sensor	Heltec V3	Medir, classificar, assinar, transmitir, mostrar no OLED	Guardar histórico, decidir confiança
Receptor de rádio	Heltec V3	Receber LoRa e repassar bytes à USB com RSSI e SNR	Verificar assinatura, decidir nada
Ponte serial	Raspberry Pi (Python)	Ler linhas da USB, converter hex em bytes, chamar o gateway	Lógica criptográfica
Gateway	Raspberry Pi (Python)	Validar, rejeitar, gravar no ledger	Falar com rádio diretamente
Ledger	SQLite	Guardar blocos encadeados e verificar a corrente	Aceitar edição silenciosa
Tela de demonstração	Raspberry Pi (Python + HTML)	Mostrar leituras, status, estado da corrente e controles de ataque	Ser necessária para o sistema funcionar

Por que o receptor não verifica a assinatura? Para manter o firmware do receptor trivial (menos risco de bug com prazo curto) e porque o gateway Python já tem a verificação testada. A verificação no ESP32 pode ser adicionada depois como opção.

4.3 Fluxo de uma leitura, passo a passo

1. A cada **intervalo de amostragem** (padrão 10 s), o nó lê temperatura e umidade.
2. Classifica: `OK`, `ALERTA` (a menos de 0,5 °C do limite) ou `ALARME` (fora de 2 a 8 °C).
3. Incrementa `seq` (contador persistente) e monta o payload de 24 bytes.
4. Calcula SHA-256 do payload e assina com a chave privada P-256, obtendo 64 bytes (r e s).
5. Atualiza o OLED e o LED.
6. Transmite os 88 bytes por LoRa.
7. O receptor recebe, anexa RSSI e SNR, e escreve na USB uma linha `RX,<rssi>,<snr>,<hex>`.
8. A ponte serial lê a linha, converte o hex em bytes e chama `Gateway.receive`.

9. O gateway: confere o tamanho (88), decodifica, procura a chave pública do `node_id`, verifica a assinatura, confere que `seq` é maior que o último aceito daquele nó e, se tudo passar, grava no ledger.
10. A tela mostra o resultado (aceito ou o motivo da rejeição) e o estado da corrente.

4.4 Decisões de arquitetura e justificativas

Decisão	Justificativa
LoRa ponto a ponto, não LoRaWAN	Não há concentrador nem servidor de rede. P2P entrega o mesmo resultado para a demonstração com muito menos risco
Receptor “burro” por USB	Reduz bug de firmware. O Python já está testado
ECDSA P-256	Suportada nativamente no mbedTLS do ESP32. Assinatura de 64 bytes cabe com folga no limite do LoRa
Assinatura bruta r e s	O firmware produz r e s separados. DER exigiria conversão extra
SQLite	Sem servidor, arquivo único, suficiente para a demonstração
Cadeia de hash replicável	Mantém a promessa do artigo sem a complexidade de consenso distribuído
Chave privada embutida no firmware	Aceitável só no protótipo, e deve ser dito

5. Hardware e ligações

5.1 Aviso importante sobre os pinos

Os números de pino abaixo são os **usuais da Heltec WiFi LoRa 32 V3**, mas **devem ser confirmados no pinout oficial da placa antes de ligar qualquer fio**, porque variam entre revisões e é fácil errar. Todos devem ficar em um único arquivo `firmware/include/pins.h`, nunca espalhados pelo código.

5.2 Pinos internos da Heltec V3 (usados pela própria placa)

Função	GPIO usual	Observação
Rádio SX1262: NSS (CS)	8	SPI do rádio
Rádio SX1262: SCK	9	
Rádio SX1262: MOSI	10	
Rádio SX1262: MISO	11	
Rádio SX1262: RESET	12	
Rádio SX1262: BUSY	13	
Rádio SX1262: DIO1	14	Interrupção do rádio
OLED: SDA	17	**Ocupa o I2C padrão**
OLED: SCL	18	
OLED: RESET	21	
Vext (alimentação externa dos periféricos)	36	Nível baixo liga. O OLED depende dele
LED branco da placa	35	
Botão PRG	0	

Como o OLED já usa o I2C padrão (`Wire`), o **SHT31-D precisa de outro barramento I2C**, usando `Wire1` em pinos livres, como GPIO 41 (SDA) e GPIO 42 (SCL). **Confirmar que esses pinos estão livres no cabeçalho da placa.**

5.3 Ligações dos periféricos

Periférico	Pino do periférico	Pino da Heltec	Observação
SHT31-D	VIN	3V3	**3,3 V, nunca 5 V**
SHT31-D	GND	GND	
SHT31-D	SDA	GPIO 41 (Wire1)	Endereço I2C 0x44 (ou 0x45 se ADDR estiver alto)
SHT31-D	SCL	GPIO 42 (Wire1)	
GPS NEO-6M	VCC	3V3 ou 5 V conforme o módulo	Conferir a placa do módulo
GPS NEO-6M	GND	GND	
GPS NEO-6M	TX	GPIO escolhido como RX do ESP32	Evitar GPIO 19 e 20, usados pela USB nativa do ESP32-S3
GPS NEO-6M	RX	GPIO escolhido como TX do ESP32	
LED de alarme	ânodo via resistor 220 Ω	GPIO livre (por exemplo 4)	Cátodo no GND

Regras de ouro do hardware:

- **Conecte a antena antes de qualquer transmissão.** Transmitir sem antena pode danificar o rádio.
- Use **cabo USB-C de dados**. Cabo só de carga faz a placa não aparecer na porta serial.
- Alimente por USB-C, power bank ou 5 V no pino correspondente. **Não** aplique 5 V ao 3V3.
- O GPS pode demorar minutos para o primeiro sinal e pode **não achar sinal dentro de prédios**. Ele é opcional (C3.3).

5.4 Montagem de bancada

- Nó: Heltec #1 + SHT31-D + LED, alimentada por power bank.
- Receptor: Heltec #2 por cabo USB ao Raspberry Pi.
- Distância inicial de teste: 1 a 2 metros, depois alguns metros e paredes, para medir RSSI.

6. Especificação do pacote (contrato central)

Esta seção é a fonte da verdade. O arquivo `src/coldchain/packet.py` a implementa. O firmware deve espelhá-la **byte a byte**.

6.1 Layout

Ordem de bytes **big-endian** (byte mais significativo primeiro). Payload de 24 bytes, assinatura de 64 bytes, total de 88 bytes.

Offset	Tam	Campo	Tipo	Descrição
0	1	`ver`	u8	Versão do formato. Hoje 1
1	2	`node_id`	u16	Identificador do nó
3	4	`seq`	u32	Contador monotônico por nó (anti-replay)
7	4	`ts`	u32	Unix time em segundos. 0 significa “desconhecido”
11	2	`temp`	i16	Temperatura em centésimos de grau Celsius
13	2	`hum`	u16	Umidade relativa em centésimos de %
15	1	`status`	u8	0 OK, 1 ALERTA, 2 ALARME
16	4	`lat`	i32	Graus vezes 10^6. `0x7FFFFFFF` significa inválido
20	4	`lon`	i32	Graus vezes 10^6. `0x7FFFFFFF` significa inválido
24	64	`sig`	bytes	ECDSA P-256 sobre SHA-256 do payload, r (32) seguido de s (32)

Exemplo de conversão: 5,25 °C vira o inteiro 525. -3,5 °C vira -350 (complemento de dois em 16 bits).

6.2 Semântica dos campos

- **`seq`:** deve ser **estritamente crescente** por `node_id`. É a defesa contra replay. Precisa **sobreviver a reinício do nó** (Seção 8.6).
- **`ts`:** o ESP32 não tem relógio de tempo real. Valor válido só se o nó obteve hora do GPS ou por configuração. Se não houver, enviar 0. O **gateway grava a hora de recebimento** (`rx_ts`) em todo bloco e trata a hora do gateway como referência. Isso exige mudar o ledger (Seção 10.2).
- **`status`:** calculado no nó, mas o gateway **recalcula** a partir de `temp` e compara. Se divergir, registra uma anomalia. O nó não é a única fonte da verdade sobre o status.
- **`lat` e `lon`:** inválidos quando o GPS não tem fix.

6.3 O que a assinatura cobre

Exatamente os 24 bytes do payload, **nada além**. O campo `sig` não faz parte do que é assinado. Qualquer alteração em qualquer byte do payload invalida a assinatura.

6.4 Versionamento

Qualquer mudança de campo exige: incrementar `ver`, atualizar esta seção, `docs/PACOTE.md`, `packet.py`, os testes e o firmware, e gerar novos vetores de teste (Seção 12.2). O gateway rejeita versão desconhecida.

6.5 Orçamento de rádio

Cálculo do tempo no ar de 88 bytes, banda de 125 kHz, taxa de codificação 4/5, preâmbulo de 8 símbolos, cabeçalho explícito, CRC ligado:

Spreading Factor	Tempo no ar aproximado	Observação
SF7	cerca de 154 ms	Mais rápido, menor alcance. **Padrão da demonstração**
SF9	cerca de 492 ms	Mais alcance. Para testes de distância
SF12	mais de 2 s	Alcance máximo. Não usar na demonstração

Esses valores são **estimativa teórica** e devem ser **medidos** na placa para o artigo. Com envio a cada 10 s e SF7, o rádio fica ocupado cerca de 1,5 por cento do tempo.

7. Criptografia e modelo de ameaças

7.1 Algoritmos

- **Hash:** SHA-256 sobre o payload de 24 bytes.
- **Assinatura:** ECDSA sobre a curva **P-256 (secp256r1, prime256v1)**.
- **Formato da assinatura:** r e s, cada um em 32 bytes big-endian, concatenados (64 bytes). **Não é DER.**
- **Chave pública:** 65 bytes, formato não comprimido `0x04 || X || Y`.

7.2 Gestão de chaves no protótipo

1. Gerar um par de chaves por nó em Python (`crypto.generate_keypair`).
2. **Chave privada:** gravar no firmware do nó (formato de 32 bytes brutos), em um arquivo `firmware/include/keys.h` **que não vai para o repositório público** (`.gitignore`).
3. **Chave pública:** cadastrar no gateway (dicionário `node_id` para chave). Pode ficar em um arquivo `keys/nodes.json`.
4. Para a demonstração, a chave é fixa. Em produção, seria provisionada em um elemento seguro (por exemplo ATECC608) e jamais exposta.

Se a chave privada vazar na bancada, **gere outro par**. Nunca reutilize chaves de teste publicadas em artigo, vídeo ou banner.

7.3 Modelo de ameaças

Ameaça	Defesa	Estado
Alterar a temperatura no caminho	Assinatura inválida	Implementado
Reenviar pacote antigo (replay)	`seq` estritamente crescente	Implementado no gateway. Persistência no nó pendente
Forjar pacote sem a chave	Assinatura inválida	Implementado
Nó desconhecido se passando por conhecido	Lista de chaves públicas confiáveis	Implementado
Editar o histórico no banco	Corrente de hash	Implementado
Apagar os últimos registros	Corrente não detecta por si só. Mitigação: replicar o hash da ponta (cabeça da corrente) em outro lugar	Discutir. Opcional
Extração da chave privada do dispositivo	Elemento seguro	Fora do escopo, citar como limitação
Interferência de rádio	Detecção de silêncio (nenhum pacote por N intervalos)	A implementar na tela
Sensor enganado fisicamente	Nenhuma	Limitação a declarar

7.4 Pontos críticos de implementação no ESP32

Estes são os lugares onde a implementação costuma falhar. Verifique cada um.

1. **Versão do mbedTLS.** O core Arduino-ESP32 2.x traz mbedTLS 2.28, o 3.x traz mbedTLS 3.x, com **APIs diferentes** (por exemplo, `mbedtls_ecdsa_sign` mudou de assinatura). Fixe a versão do core no `platformio.ini` e confira a documentação dessa versão.
2. **Aleatoriedade.** ECDSA precisa de um número aleatório por assinatura. Um número repetido ou previsível **revela a chave privada**. Prefira a variante determinística (`mbedtls_ecdsa_sign_det_ext`, RFC 6979) quando disponível, ou garanta que o gerador de números aleatórios do ESP32 esteja alimentado.
3. **Formato da saída.** `mbedtls_ecdsa_sign` devolve `r` e `s` como inteiros grandes (`mbedtls_mpi`). Escreva cada um com `mbedtls_mpi_write_binary` em exatamente 32 bytes. **Não** use as funções que produzem DER.
4. **Alternativa mais simples:** a biblioteca **micro-ecc** (`uECC_sign` com `uECC_secp256r1()`) já entrega `r` e `s` em 64 bytes. É uma opção razoável se o mbedTLS atrapalhar. **Decisão pendente sobre a curva:** o artigo usou secp256k1 e a micro-ecc oferece `uECC_secp256k1()`. Usar secp256k1 elimina uma diferença com o artigo, mas obriga a trocar `CURVE` em `crypto.py`, regenerar `tests/vectors.json` e conferir se o mbedTLS do core em uso tem a curva habilitada (com micro-ecc isso não importa). Recomendação: **decidir uma vez e registrar**; sem decisão contrária, o padrão é P-256 declarada.
5. **Prova de compatibilidade antes de integrar.** Gere a assinatura no ESP32 de um payload fixo e confirme que o Python a verifica (Seção 12.2). Faça isso **antes** de ligar o rádio.
6. **Tempo de assinatura.** Meça e registre. Não afirme um número sem medir.

8. Firmware do nó sensor

Esta seção descreve o que o firmware do nó deve fazer. Os trechos de código são **esqueletos para orientar**, não foram compilados nem testados em hardware. Confirme cada API na documentação da versão que for usada.

8.1 Ferramentas e bibliotecas

Item	Escolha	Observação
Ambiente	**PlatformIO** (VS Code) ou Arduino IDE 2	PlatformIO fixa versões e é reprodutível
Plataforma	`espressif32` com framework `arduino`	Fixar a versão do core
Placa	Heltec WiFi LoRa 32 V3 (`heltec_wifi_lora_32_V3` no PlatformIO)	Confirmar o nome na lista de placas
Rádio	**RadioLib** (jgromes)	Suporta SX1262
Atalho de bring-up	biblioteca `heltec_unofficial` (ropg)	Resolve pinos, OLED e rádio da V3. Útil para o primeiro teste. Confirmar disponibilidade
Sensor	`Adafruit SHT31` (com `Adafruit BusIO`)	Ou Sensirion. Endereço 0x44
OLED	`Adafruit SSD1306` e `Adafruit GFX`, ou `U8g2`	Controlador SSD1306 de 128x64
GPS	`TinyGPSPlus`	Camada 3
Cripto	**mbedTLS** do próprio core, ou `micro-ecc`	Seção 7.4
Persistência	`Preferences` (NVS do ESP32)	Para o `seq`

Esqueleto de `platformio.ini` (ajustar versões):

```
[env:heltec_v3_node]
platform = espressif32@6.x        ; fixar a versão exata que funcionar
board = heltec_wifi_lora_32_V3
framework = arduino
monitor_speed = 115200
build_flags =
    -DNODE_ID=1
    -DLORA_FREQ_MHZ=915.0
lib_deps =
    jgromes/RadioLib
    adafruit/Adafruit SHT31 Library
    adafruit/Adafruit SSD1306
    adafruit/Adafruit GFX Library
```

8.2 Estrutura de pastas do firmware

```
firmware/
  node/                  # imagem do nó sensor
    platformio.ini
    src/main.cpp
    src/sensor.cpp/.h    # abstração do sensor (SHT31)
    src/radio.cpp/.h     # LoRa via RadioLib
    src/signer.cpp/.h    # SHA-256 e ECDSA
    src/packet.cpp/.h    # monta os 24 bytes
    src/display.cpp/.h   # OLED
    src/storage.cpp/.h   # seq persistente (NVS)
  receiver/              # imagem do receptor
    platformio.ini
    src/main.cpp
  include/
    pins.h               # todos os pinos
    config.h             # intervalo, limites, parâmetros LoRa
    keys.h               # chave privada (fora do git)
  test_vectors/          # vetores gerados pelo Python
```

8.3 Ciclo principal (máquina de estados)

```
BOOT -> INIT_HW -> CARREGA_SEQ -> LOOP:
   LEITURA   : lê SHT31 (se falhar, marca erro e tenta de novo)
   DECISÃO   : classifica (OK, ALERTA, ALARME)
   PACOTE    : incrementa seq, grava seq na NVS, monta 24 bytes
   ASSINA    : SHA-256 + ECDSA P-256 -> 64 bytes
   MOSTRA    : OLED e LED
   TRANSMITE : LoRa, 88 bytes
   ESPERA    : até completar o intervalo de amostragem
```

Regras:

- O intervalo padrão é **10 s**, configurável em `config.h`. Para testes de bancada, pode ser 2 s.
- **Gravar o `seq` na NVS antes de transmitir.** Se o nó reiniciar entre gravar e transmitir, o `seq` pula um número, o que é aceitável. O inverso (transmitir e não gravar) causaria `seq` repetido após reinício, e o gateway rejeitaria como replay.
- O loop **não pode travar** esperando o sensor. Use tempo limite e registre o erro no OLED.
- O watchdog do ESP32 deve permanecer ativo.

8.4 Abstração do sensor

Para permitir trocar o SHT31-D por outro sensor (equivalente declarado), defina uma interface mínima:

```
struct Medida { float temp_c; float hum_pct; bool ok; };
Medida sensor_ler();        // implementação do SHT31 em sensor.cpp
```

Qualquer substituto (DS18B20, DHT22) só precisa implementar `sensor_ler`. Se o sensor não medir umidade, enviar 0 e documentar.

8.5 Rádio LoRa (parâmetros)

Parâmetro	Valor	Motivo
Frequência	**915,0 MHz**	Faixa legal no Brasil (915 a 928 MHz). A placa aceita 863 a 928
Largura de banda	125 kHz	Padrão
Spreading Factor	7 (demonstração), 9 (testes de alcance)	Seção 6.5
Coding rate	4/5	Padrão
Palavra de sincronismo	0x12 (privada)	Evita misturar com redes públicas
Potência	14 dBm no início	Aumentar só se necessário. Respeitar limites da Anatel
Preâmbulo	8 símbolos	Padrão
CRC	Ligado	Descarta pacotes corrompidos no rádio

Os parâmetros do nó e do receptor devem ser idênticos. Diferença em qualquer um deles faz o receptor não ouvir nada. Mantenha-os em um único `config.h` compartilhado.

Esqueleto de inicialização (confirmar API e parâmetros da versão do RadioLib):

```
#include <RadioLib.h>
SX1262 radio = new Module(LORA_NSS, LORA_DIO1, LORA_RST, LORA_BUSY);


// SPI.begin(LORA_SCK, LORA_MISO, LORA_MOSI, LORA_NSS);
// int estado = radio.begin(915.0, 125.0, 7, 5, 0x12, 14, 8);
// radio.setCRC(true);
// envio:  radio.transmit(pacote, 88);
```

Ponto de falha mais comum no primeiro bring-up da Heltec V3: configuração do **TCXO** (tensão e uso do DIO3) e do controle do **chave RF (DIO2)** do SX1262. Se `radio.begin` retornar código de erro, o primeiro suspeito é isso. Os exemplos oficiais da Heltec e os exemplos de RadioLib para a V3 mostram os valores corretos. Por isso a recomendação de começar com a biblioteca `heltec_unofficial` ou com o exemplo oficial, e só depois migrar.

8.6 Persistência do contador `seq`

Problema: se o nó reinicia e o `seq` volta a 1, o gateway rejeita tudo como replay até o `seq` passar do último aceito.

Solução: guardar o `seq` na **NVS** (`Preferences`). No boot, ler; a cada pacote, incrementar e gravar. O desgaste da flash é desprezível para o volume da demonstração (milhares de gravações).

Plano de emergência para a demonstração: o gateway Python terá uma opção de linha de comando `--reset-node <id>` que registra um bloco especial de “ressincronização” e aceita um novo início de `seq`.

Deve ficar **desligada por padrão**, para não enfraquecer a proteção, e a existência dela deve ser dita se perguntarem.

8.7 Assinatura no nó

Entrada: 24 bytes do payload. Saída: 64 bytes. Fluxo:

1. Calcular SHA-256 do payload (mbedTLS: `mbedtls_sha256`).
2. Assinar o hash com a chave privada (ver 7.4).
3. Escrever r e s em 32 bytes cada, big-endian.
4. Anexar ao payload.

Medição obrigatória: registrar o tempo (`micros()` antes e depois) de uma assinatura, repetir pelo menos 100 vezes e guardar média, mínimo, máximo e desvio. Esse número vai para o artigo no lugar dos 0,59 ms do PC.

8.8 Interface local (OLED e LED)

O OLED mostra, em linhas curtas:

```
NO 1   seq 128
T 5.25 C   U 43%
STATUS: OK
TX ok  sig 18ms
```

O LED acende fixo em ALARME e pisca em ALERTA. O botão PRG pode alternar telas (leitura, estatísticas, chave pública resumida).

8.9 Critério de aceite do firmware do nó

- Lê o sensor e mostra no OLED.
- LED reage aos limites (testar aquecendo o sensor com a mão e resfriando com gelo em saquinho).
- Gera pacote de 88 bytes **idêntico em formato** ao do Python.
- Um pacote gerado pelo nó, capturado pela serial do receptor, **passa em `Gateway.receive` sem alteração**.
- Reinício do nó não gera replay (o `seq` continua).

9. Firmware do receptor e ponte serial

9.1 O que o receptor faz

Ouvir o rádio o tempo todo e, a cada pacote recebido, escrever uma linha de texto na USB. Nada mais. Parâmetros de rádio iguais aos do nó.

9.2 Esqueleto do receptor

```
// setup: inicia serial 115200, rádio com os mesmos parâmetros, radio.startReceive()
// loop:  se chegou pacote:
//          len = radio.getPacketLength(); radio.readData(buf, len);
//          rssi = radio.getRSSI(); snr = radio.getSNR();
//          imprime "RX,<rssi>,<snr>,<hex de len bytes>\n"
//          radio.startReceive();
```

Se o comprimento recebido não for 88, imprimir mesmo assim (o gateway rejeita com `tamanho_invalido`), porque isso é útil para diagnóstico.

9.3 Protocolo serial (contrato entre receptor e Raspberry Pi)

Uma linha de texto por evento, terminada em `\n`, 115200 baud, 8N1.

Linha	Significado
`RX,<rssi>,<snr>,<hex>`	Pacote recebido. `rssi` em dBm (inteiro), `snr` em dB (decimal), `hex` em minúsculas sem espaços
`INFO,<texto>`	Mensagem informativa (boot, parâmetros)
`ERR,<código>,<texto>`	Erro do rádio

Exemplo: `RX,-63,9.5,01000100000001...` (176 caracteres hexadecimais para 88 bytes).

A ponte Python **ignora linhas que não começam com esses prefixos** e **nunca deve cair** por causa de uma linha malformada.

9.4 Ponte serial em Python

Arquivo novo: `src/coldchain/serial_bridge.py`. Responsabilidades:

- Abrir a porta (`/dev/ttyACM0` ou `/dev/ttyUSB0`, configurável), com reconexão automática se a Heltec for desligada.
- Ler linha a linha, interpretar o prefixo, converter o hex em bytes.
- Chamar `Gateway.receive(pacote)` e entregar o resultado e o RSSI e SNR à camada de apresentação.
- Registrar tudo em log com data e hora.
- Ter um **modo simulado** (`--simulate`) que gera linhas `RX,...` a partir do `SimNode`, para desenvolver e ensaiar **sem hardware**.

Dependência nova: `pyserial`.

9.5 Instalação no Raspberry Pi

- Raspberry Pi OS (Lite serve), Python 3.10 ou superior (se o sistema vier com 3.9, instalar via pyenv ou usar a versão mais nova disponível e revisar a sintaxe `int | None`, usada no código).
- Usuário no grupo `dialout` para acessar a porta serial: `sudo usermod -aG dialout $USER`.

- Ambiente virtual: `python -m venv .venv && . .venv/bin/activate && pip install cryptography pyserial fastapi uvicorn`.
- Serviço `systemd` para o gateway subir sozinho (opcional, mas recomendado para a demonstração).

10. Gateway, ledger e tela de demonstração

10.1 Validações do gateway (ordem fixa)

O método `Gateway.receive(pacote)` aplica as verificações nesta ordem. A ordem importa porque as mais baratas vêm primeiro e o resultado informa a causa exata.

Ordem	Verificação	Resultado se falhar
1	Tamanho igual a 88 bytes	`tamanho_invalido`
2	Versão conhecida e `status` válido	`payload_invalido`
3	`node_id` cadastrado	`no_desconhecido`
4	Assinatura válida com a chave do nó	`assinatura_invalida`
5	`seq` maior que o último aceito do nó	`replay`
6	Tudo certo: grava no ledger	`aceito`

Evoluções previstas no gateway:

- Recalcular `status` a partir de `temp` e comparar com o campo recebido. Divergência vira anotação de anomalia no bloco.
- Detectar **silêncio**: se um nó cadastrado não envia nada por 3 intervalos, gerar um evento `silencio`.
- Aceitar RSSI e SNR e guardá-los junto ao bloco (informação de qualidade do enlace).

10.2 Ledger (esquema recomendado)

O ledger atual guarda `idx`, `node_id`, `seq`, `payload`, `sig`, `prev_hash`, `hash`. Para a tela e para as medições, acrescentar colunas **sem alterar o cálculo do hash**, que continua sendo `SHA256(prev_hash || payload || sig)`:

```
CREATE TABLE blocks (
  idx        INTEGER PRIMARY KEY AUTOINCREMENT,
  node_id    INTEGER NOT NULL,
  seq        INTEGER NOT NULL,
  payload    BLOB    NOT NULL,   -- 24 bytes
  sig        BLOB    NOT NULL,   -- 64 bytes
  prev_hash  BLOB    NOT NULL,
  hash       BLOB    NOT NULL,
  rx_ts      INTEGER,            -- hora de recebimento no gateway (unix)
  rssi       INTEGER,
  snr        REAL,
  temp_c     REAL,               -- cópia decodificada, só para consulta
  status     INTEGER,
  anomaly    TEXT                -- ex.: 'status_divergente'
);
CREATE TABLE events (            -- eventos fora da corrente
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  rx_ts INTEGER, kind TEXT, detail TEXT
);
```

Observações:

- As colunas `rx_ts`, `rssi`, `snr`, `temp_c`, `status`, `anomaly` **não entram no hash**. Elas são conveniência. A fonte da verdade é `payload` e `sig`. Por isso `verify_chain` deve também **reconferir a assinatura** de cada bloco (com as chaves públicas) além do hash, e **comparar** as colunas decodificadas com o payload, apontando divergência.
- Rejeições (assinatura inválida, replay, nó desconhecido) vão para `events`, **não** para `blocks`. Assim a corrente só contém dados válidos, e a tela ainda consegue mostrar os ataques.

10.3 Verificação da corrente

`verify_chain` deve devolver não só “ok ou não”, mas **qual bloco** falhou e **por quê**: `hash_quebrado`, `prev_hash_diferente`, `assinatura_invalida` ou `coluna_divergente`. Na demonstração, a tela destaca em vermelho o bloco apontado.

Limitação a conhecer: apagar os **últimos** blocos não quebra a corrente. A mitigação é publicar a “cabeça” (o último hash) em outro lugar, por exemplo mostrando na tela e no OLED do receptor, ou enviando por MQTT. Explique isso como trabalho futuro.

10.4 Tela de demonstração (web local)

Aplicativo **FastAPI** servindo uma página única em HTML, JavaScript puro e CSS, sem dependência de CDN (a competição pode ter Wi-Fi ruim; deixe todos os arquivos locais). Gráfico com **Chart.js** salvo localmente ou desenhado em canvas simples.

Conteúdo da tela, de cima para baixo:

1. **Cabeçalho:** nome do projeto e estado da conexão serial (verde, vermelho).
2. **Cartão do nó:** última temperatura, umidade, status com cor (OK verde, ALERTA amarelo, ALARME vermelho), `seq`, RSSI e SNR, segundos desde o último pacote.
3. **Gráfico** de temperatura por tempo, com a faixa de 2 a 8 °C sombreada.
4. **Lista de eventos** em tempo real: aceitos e rejeitados, com o motivo.
5. **Painel da corrente:** número de blocos, hash da cabeça (primeiros 12 caracteres), botão "Verificar corrente" e resultado.
6. **Painel de ataque (modo demonstração):**
   - "Adulterar pacote": envia ao gateway uma cópia do último pacote com a temperatura alterada.
   - "Reenviar pacote": reenvia o último pacote (replay).
   - "Adulterar o banco": edita diretamente um bloco antigo no SQLite.
   - "Restaurar demo": reinicia ledger e contadores para a próxima rodada.

Os botões de ataque ficam atrás de uma bandeira `--demo-controls` e **não existem** no modo normal.

10.5 API HTTP (contrato da tela)

Método e rota	Resposta
`GET /api/status`	Estado da serial, nós conhecidos, tamanho e cabeça da corrente
`GET /api/readings?limit=100`	Últimas leituras aceitas (JSON)
`GET /api/events?limit=100`	Últimos eventos, incluindo rejeições
`GET /api/chain/verify`	`{ok, bloco_invalido, motivo}`
`POST /api/demo/tamper`	Injeta pacote adulterado e devolve o resultado
`POST /api/demo/replay`	Reenvia o último pacote aceito
`POST /api/demo/corrupt-db`	Altera um bloco no banco
`POST /api/demo/reset`	Reinicia o estado da demonstração
`GET /ws` (WebSocket, opcional)	Empurra novos eventos para a página

Se o WebSocket complicar, a página pode consultar `GET /api/readings` e `GET /api/events` a cada 1 s. **Prefira o simples**.

10.6 Configuração

Arquivo `config.toml` (ou variáveis de ambiente) com: porta serial, baud, caminho do banco, arquivo de chaves públicas, limites térmicos (`low`, `high`, `margin`), intervalo esperado do nó (para detectar silêncio), porta da tela. Nenhum valor mágico espalhado pelo código.

11. Camada 3: extras

Só começar depois que a camada 2 estiver **funcionando e ensaiada**.

11.1 MQTT local

Publicar cada leitura aceita em um broker **Mosquitto** local.

- Tópico: `coldchain/<node_id>/reading`.
- Mensagem (JSON): `{"node":1,"seq":128,"ts":...,"temp_c":5.25,"hum_pct":43.5,"status":"OK","rssi":-63,"snr":9.5,"hash":"ab12..."}`.
- Tópico de eventos: `coldchain/<node_id>/event` para rejeições e silêncio.
- Biblioteca Python: `paho-mqtt`.

Valor: mostra o caminho para a nuvem e permite um painel externo (Grafana ou Node-RED) se houver tempo.

11.2 AWS IoT Core e DynamoDB

O artigo cita AWS IoT Core. Se for implementar:

- Criar uma *thing* para o gateway (não para cada nó, já que quem fala com a nuvem é o gateway).
- Certificado X.509 do gateway, política mínima de publicação no tópico `coldchain/#`.
- Regra de IoT para gravar no **DynamoDB** (tabela por `node_id` e `seq`).
- Conexão MQTT sobre TLS (porta 8883) com `paho-mqtt` ou o SDK AWS IoT para Python.
- **Cuidado com custo e credenciais.** Nunca coloque certificados ou chaves no repositório. Use conta com limite de gasto.

Se não houver tempo, **declare como trabalho futuro**. A demonstração local é suficiente e mais confiável.

11.3 GPS NEO-6M

- Ligar por UART, ler NMEA com `TinyGPSPlus`.
- Preencher `lat` e `lon` do pacote apenas com **fix válido**. Sem fix, usar `0x7FFFFFFF`.
- Quando houver fix, usar o horário do GPS para preencher `ts` (unix time).
- Testar **fora do prédio**, com visão do céu. Dentro de casa o resultado costuma ser “sem fix”, e isso é normal.
- Na demonstração, se o GPS não pegar sinal, mostrar “GPS sem fix” no OLED e seguir. Nunca depender dele.

11.4 Energia (discussão, sem implementação)

O artigo cita energia solar. Na prática, medir o consumo do nó com um multímetro ou medidor USB em dois modos (ativo transmitindo e *deep sleep* entre amostras) e **apresentar a medição como estimativa de autonomia**. Isso rende resultado para o artigo com baixo custo de tempo. Não prometer autonomia sem medir.

12. Plano de testes

12.1 Testes automatizados (já existentes e a ampliar)

O repositório tem 16 testes. Rodar com `python -m unittest discover -s tests`. Casos cobertos hoje: tamanhos do pacote, ida e volta de campos, GPS inválido, temperatura negativa, classificação, assinatura e verificação, chave pública bruta, PEM, aceitação, payload e assinatura adulterados, replay, tamanho errado, nó desconhecido, chave errada, detecção de adulteração na corrente.

A acrescentar:

- Ponte serial com linhas malformadas (não pode cair).
- `verify_chain` reconferindo assinaturas e colunas decodificadas.
- Detecção de silêncio.
- Divergência de `status`.
- Vetores dourados (12.2).

12.2 Vetores dourados de interoperabilidade

São o teste mais importante do projeto, porque provam que firmware e Python falam a mesma língua **antes** de ligar o rádio. O arquivo `tests/vectors.json` (já gerado no repositório) contém:

- Uma chave privada de teste (32 bytes em hexadecimal) e a pública correspondente (65 bytes).
- Alguns payloads de 24 bytes em hexadecimal, com os campos decodificados.
- Para cada um, uma assinatura válida (64 bytes) e o pacote completo de 88 bytes.
- Casos negativos: payload com 1 bit alterado e assinatura com 1 bit alterado, ambos devendo ser rejeitados.

Como usar:

1. **Python verifica os vetores** (teste automatizado já incluído).
2. **Firmware verifica os vetores:** embutir a chave pública e um dos pacotes no firmware de teste, verificar com mbedTLS, imprimir “OK” ou “FALHA” na serial.
3. **Firmware assina e o Python verifica:** gravar a chave de teste no firmware de teste, assinar o payload fixo e imprimir o pacote. O Python deve aceitar. (A assinatura ECDSA não é determinística entre implementações, então compare a **validade**, não a igualdade de bytes.)

A chave dos vetores é **pública e só serve para teste**. Jamais use na demonstração.

12.3 Testes de bancada no hardware

Teste	Procedimento	Critério de aceite
T1	Gravar firmware mínimo (piscar LED e escrever na serial) em cada Heltec	As duas placas respondem
T2	Ler o SHT31-D e mostrar no OLED	Temperatura plausível (ambiente)
T3	Variar a temperatura (mão e gelo em saquinho)	Valor responde e LED muda
T4	Verificar o vetor dourado no ESP32	“OK” na serial
T5	ESP32 assina payload fixo, Python verifica	Aceito
T6	LoRa: nó transmite, receptor imprime a linha `RX, ...`	Pacote de 88 bytes íntegro
T7	Ponte serial entrega ao gateway	`aceito` na tela
T8	Adulterar 1 bit no caminho (botão de demo)	`assinatura_invalida`
T9	Reiniciar o nó e continuar enviando	Sem `replay`
T10	Reenviar pacote	`replay`
T11	Distância e obstáculos (1 m, 5 m, parede)	Anotar RSSI, SNR e perdas
T12	Funcionar 30 minutos seguidos	Sem travar e sem perda sistemática

12.4 Medições para o artigo

Registrar em planilha, com data, versão do firmware e condições:

- Tempo de assinatura no ESP32 (média, mínimo, máximo, desvio, n de pelo menos 100).
- Tempo de verificação no gateway Python.
- Tempo no ar do pacote (teórico e medido) em SF7 e SF9.
- Taxa de entrega (pacotes aceitos sobre pacotes enviados) em 3 cenários de distância.
- RSSI e SNR por cenário.
- Taxa de detecção de adulteração (por exemplo 50 de 50, agora com hardware).
- Consumo do nó (opcional, Seção 11.4).

Esses números **substituem ou complementam** os da simulação, e a diferença para o PC deve ser comentada.

13. Artigo versus demonstração

Esta tabela evita o maior risco da apresentação: mostrar algo diferente do que o artigo afirma. Preencher e **atualizar a cada semana**.

Item do artigo	Estado na demonstração	Como declarar
ESP32-WROOM-32 com SX1276 (TTGO), Tabela 1 “Planejado”	Heltec V3 (ESP32-S3, SX1262)	“Trocamos a placa por disponibilidade e custo”
SHT31-D	Implementado	“Sensor real, medido”
ECDSA com SHA-256	Firmware, curva P-256 (ou secp256k1 se decidido)	“A simulação usou secp256k1, o protótipo usa P-256”
micro-ecc	mbedTLS ou micro-ecc	Informar a biblioteca realmente usada
Resultados 14/14, 50/50, 0,59 ms	Reproduzidos com `tools/repro_artigo.py`; latência medida de novo no ESP32	“0,59 ms foi em PC. No ESP32 mediu-se X ms”
Ciclo de 60 s	Configurável; demonstração em intervalo menor	Declarar o intervalo usado
Payload JSON	Pacote binário de 88 bytes	“Binário por caber no LoRa e reduzir tempo no ar”
Bloco encadeado no nó	Corrente no gateway (SQLite)	“A corrente é verificada no gateway”
LoRaWAN, GSM/4G, MQTT	LoRa ponto a ponto	“Emulamos o enlace. Concentrador LoRaWAN e GSM são trabalho futuro”
Store-and-forward em LittleFS	Camada 3, se sobrar tempo	Conforme o que estiver pronto
MPU-6050, DS3231, Reed Switch, bateria 5.000 mAh	Não comprados, não implementados	“Especificados no artigo, fora do protótipo desta fase”
GPS NEO-6M	Opcional, depende de sinal	“Funciona com céu aberto” ou “trabalho futuro”
Autonomia superior a 72 h	Não medida	“Especificação, não medição”
AWS IoT Core	Implementado ou declarado futuro	Conforme o que estiver pronto
Câmara fria (trabalhos futuros)	Teste simples com caixa térmica e gelo, se houver tempo	“Teste de bancada, não é câmara fria validada”
Rota Fortaleza a Juazeiro do Norte	Não realizada	“Cenário alvo, não testado em campo”

13.1 Tabela 3 do artigo contra a demonstração

A Tabela 3 do artigo lista as hipóteses simplificadoras da simulação e a evolução proposta. A demonstração concretiza várias delas, e isso é argumento de **potencial de evolução** para a banca.

Evolução proposta no artigo	O que a demonstração entrega
Sensor SHT31-D físico, com ruído real	Entrega, com leitura real no ESP32
Medir latência real de transmissão	Entrega, com tempo no ar e RSSI/SNR medidos em LoRa (ponto a ponto, não LoRaWAN)
Chaves e assinatura no ESP32	Entrega, com a chave embutida no protótipo
Cenário de ataque coordenado	Parcial: adulteração, replay e edição do banco
Motor de validação em nuvem	Só se a Camada 3 for feita
Limiar por categoria de insumo	Limiares configuráveis no gateway
Modelar consumo de bateria	Fora desta fase, a menos que haja medição

Regra: **quando houver dúvida, declare a diferença**. Honestidade sobre o que é medido, simulado ou futuro costuma pesar a favor, pois os critérios valorizam qualidade do texto e potencial de evolução.

14. Roteiro da demonstração e material

14.1 Roteiro de 5 minutos no estande

1. **Contexto (30 s):** o problema da cadeia de frio e a ideia (assinar na origem).
2. **Nó em funcionamento (60 s):** mostrar o dispositivo medindo, o OLED e o LED. Colocar a mão no sensor para subir a temperatura até o alarme.
3. **Chegada ao gateway (45 s):** na tela, o pacote aceito, com RSSI e SNR.
4. **Ataque 1, adulteração (45 s):** apertar “Adulterar pacote”. A tela mostra `assinatura_invalida`. Explicar o lacre matemático.
5. **Ataque 2, replay (30 s):** apertar “Reenviar pacote”. A tela mostra `replay`.
6. **Corrente (60 s):** “Verificar corrente” (verde). Apertar “Adulterar o banco” e verificar de novo, apontando o bloco quebrado em vermelho.
7. **Resultados e limites (30 s):** mostrar a tabela de medições e falar com franqueza das limitações da Seção 2.3.

14.2 Perguntas prováveis e respostas curtas

- *“Isso é blockchain?”* Não. É uma cadeia de hash replicável. Não há consenso distribuído.
- *“E se roubarem a chave?”* No protótipo, a chave está no firmware. Em produção seria guardada em um elemento seguro.
- *“Por que não LoRaWAN?”* Emulamos o enlace com LoRa ponto a ponto. LoRaWAN exige concentrador e servidor de rede, listado como trabalho futuro.
- *“A assinatura impede alguém de aquecer o sensor?”* Não. Ela protege o caminho do dado, não a medição. Para isso seria preciso selagem física.
- *“Funciona sem internet?”* Sim, a demonstração é toda local.

14.3 Material multimídia

Já produzidos: artigo, banner e roteiro de vídeo. A atualizar conforme o hardware real: fotos e vídeos de cada etapa de bancada (também servem de evidência de “resultados obtidos até o momento”), e a figura do diagrama da arquitetura.

15. Plano de implementação (para o Claude Code)

Esta é a lista de trabalho em ordem. Cada tarefa tem **entrega** e **critério de aceite**. Marque como feita só quando o critério passar. Em caso de dúvida sobre pino, API ou parâmetro de rádio, **pergunte ou consulte a documentação oficial**, não adivinhe.

Bloco A, sem hardware (pode começar hoje)

Id	Tarefa	Entrega	Aceite
A1	Ler este documento e o `CLAUDE.md`, rodar os testes e a demo	Relato curto do que entendeu	16 testes passam e `run_demo.py` roda
A2	Gerar vetores dourados (`tests/vectors.json`) e o teste que os valida	Arquivo e teste	Teste passa, com os casos negativos rejeitados
A3	Estender o ledger (colunas, tabela `events`, `verify_chain` detalhado)	`ledger.py` e testes	Corrente reconfere hash, assinatura e colunas, e aponta o motivo
A4	Estender o gateway (recalcular status, silêncio, RSSI e SNR, eventos de rejeição)	`gateway.py` e testes	Cada rejeição vira evento e não entra na corrente
A5	Ponte serial com modo `--simulate`	`serial_bridge.py` e testes	Linhas malformadas não derrubam. Modo simulado alimenta o gateway
A6	Configuração em `config.toml` e arquivo de chaves públicas	`config.py`, `keys/`	Nenhum valor mágico no código
A7	Tela web com API (Seção 10) e modo `--demo-controls`	`webapp.py`, `static/`	Demonstração A a E roda no modo simulado em menos de 5 minutos
A8	Medir tempo de verificação no Python e gerar relatório	Script e CSV	Média, mínimo, máximo e desvio

Bloco B, quando houver hardware

Id	Tarefa	Entrega	Aceite
B1	Bring-up das duas Heltec (T1)	Firmware mínimo	Ambas respondem
B2	Sensor e OLED no nó (T2, T3)	`sensor.cpp`, `display.cpp`	Leitura plausível e reativa
B3	Vetor dourado no ESP32 (T4, T5)	Firmware de teste	`OK` na serial e Python aceita assinatura do ESP32
B4	LoRa nó para receptor (T6)	`radio.cpp`, firmware do receptor	Linha `RX,...` com 88 bytes íntegros
B5	Nó completo com `seq` na NVS (T9)	`main.cpp`, `storage.cpp`	Reinício sem replay
B6	Integração ponta a ponta (T7, T8, T10)	Tudo ligado	Demonstração A a E com hardware real
B7	Medições para o artigo (12.4)	Planilha	Todos os itens preenchidos
B8	Ensaio de 30 minutos e T11, T12	Relatório	Sem travar

Bloco C, extras (só se B estiver pronto)

C1 MQTT local. C2 GPS. C3 AWS IoT Core. C4 Medição de consumo.

Regras de trabalho

- Um commit pequeno por tarefa, com mensagem clara.
- **Nunca** commitar `keys.h`, chaves privadas, certificados ou credenciais.
- Alterou o formato do pacote? Atualize `docs/PACOTE.md`, o código, os testes, os vetores e o firmware **na mesma entrega**.
- Código em inglês, comentários e documentação em português.
- Não adicionar dependência sem necessidade. Dependências atuais: `cryptography`. Previstas: `pyserial`, `fastapi`, `uvicorn`. Opcionais: `paho-mqtt`.
- Nunca fingir resultado. Se um teste de hardware não foi feito, dizer que não foi feito.

16. Cronograma

Hoje é 19/10/2026. A competição começa em 23/11/2026.

Período	Meta	Critério
19 a 22/10	Bloco A1 a A4. Decisão sobre o pedido (checagem em 21/10)	Gateway estendido e testado. Pedido mantido ou cancelado com compra nacional
23 a 28/10	Bloco A5 a A7	Demonstração simulada completa
29/10 a 31/10	Bloco B1 a B4 (se as peças chegaram)	LoRa nó para receptor
1 a 7/11	B5, B6	Assinatura no firmware e ponta a ponta
5/11	**Data de corte**	Se o hardware não funciona, ativar o plano B
8 a 12/11	B7, B8	Demonstração fechada e medições
12 a 22/11	Ensaio, banner, vídeo, tabela “Artigo × demonstração”	Equipe consegue apresentar sem ajuda
23 a 29/11	Fase Nacional	Presença da equipe é obrigatória

17. Riscos e plano B

Risco	Probabilidade	Impacto	Mitigação
Peças do AliExpress atrasam	Alta (envio da China)	Alto	Checagem em 21/10. Compra nacional da Heltec. Bloco A independe de hardware
Heltec com defeito ou TCXO mal configurado	Média	Alto	Duas placas. Começar pelos exemplos oficiais
ECDSA no ESP32 não compila ou diverge	Média	Alto	Vetores dourados antes do rádio. Alternativa micro-ecc
Incompatibilidade de versões de bibliotecas	Média	Médio	Fixar versões no `platformio.ini`
SHT31-D sem solda ou defeituoso	Média	Médio	Sensor reserva. Substituto equivalente declarado
GPS sem sinal	Alta	Baixo	Opcional
Wi-Fi ruim no local	Alta	Médio	Tudo local, sem CDN, sem nuvem na demonstração
Tempo insuficiente	Média	Alto	Cortar de trás para frente (Seção 3)
Equipamento danificado no transporte	Baixa	Alto	Peças de reserva, cabos extras, cópia do projeto em outro computador

Plano B (se o hardware não funcionar até 5/11): apresentar a simulação completa em Python com a tela de demonstração (Bloco A), o vídeo, o banner e a tabela "Artigo × demonstração", **dizendo claramente** que o protótipo físico está em desenvolvimento. Isso é permitido pelas regras lidas (resultados até o momento e próximos passos) e é muito melhor do que apresentar algo que falha ao vivo sem explicação.

Plano C (se o hardware funcionar parcialmente): mostrar o que funciona (por exemplo, sensor, assinatura e envio) e usar o simulador para a parte que falta, **identificando** na tela o que é real e o que é simulado.

18. Stack consolidada

Camada	Tecnologia	Versão ou observação
Linguagem do gateway	Python	3.10 ou superior
Criptografia (Python)	`cryptography`	Curva SECP256R1
Banco	SQLite	Embutido no Python
Serial	`pyserial`	Ponte com o receptor
Web	FastAPI e Uvicorn	Tela de demonstração
Front-end	HTML, JavaScript puro, CSS, Chart.js local	Sem CDN
Mensageria (opcional)	Mosquitto e `paho-mqtt`	Camada 3
Nuvem (opcional)	AWS IoT Core e DynamoDB	Camada 3
Firmware	C++ (framework Arduino no ESP32-S3) via PlatformIO	Fixar versões
Rádio	RadioLib com SX1262	915 MHz
Sensor	`Adafruit SHT31`	I2C, `Wire1`
Tela do nó	`Adafruit SSD1306` ou `U8g2`	OLED integrado
GPS (opcional)	`TinyGPSPlus`	UART
Cripto (firmware)	mbedTLS ou micro-ecc	Seção 7.4
Persistência no nó	`Preferences` (NVS)	Contador `seq`
Testes	`unittest`	Já em uso
Sistema do gateway	Raspberry Pi OS	Com `systemd` opcional

19. Glossário

- **Nó sensor:** o dispositivo dentro da caixa térmica que mede e assina.
- **Gateway:** o conjunto que recebe, valida e grava (receptor de rádio mais Raspberry Pi).
- **LoRa:** modulação de rádio de longo alcance e baixa potência.
- **LoRaWAN:** protocolo de rede construído sobre o LoRa, com concentradores e servidor de rede. Não usado no protótipo.
- **ECDSA:** algoritmo de assinatura digital por curvas elípticas.

- **P-256 (secp256r1):** curva elíptica de 256 bits, usada neste projeto. A simulação do artigo usou secp256k1.
- **SHA-256:** função de hash que resume dados em 32 bytes.
- **Hash:** impressão digital de um dado. Mudar 1 bit muda tudo.
- **Cadeia de hash (ledger):** sequência de registros em que cada um inclui o hash do anterior.
- **Replay:** reenvio de um pacote legítimo antigo.
- **seq:** contador crescente usado para detectar replay.
- **RSSI:** intensidade do sinal recebido, em dBm.
- **SNR:** relação sinal-ruído, em dB.
- **NVS:** memória não volátil do ESP32, usada para guardar o `seq`.
- **TCXO:** oscilador compensado do rádio, ponto de atenção na configuração da Heltec V3.
- **Spreading Factor (SF):** parâmetro do LoRa. Maior SF dá mais alcance e mais tempo no ar.
- **Tempo no ar (airtime):** duração da transmissão de um pacote.
- **MNR:** Mostra Nacional de Robótica, da RoboCup Brasil.

20. Referências e fontes de consulta

- Página oficial da MNR e manuais: mnr.robocup.org.br (documentos, FAQ, formatos de submissão, manual de 2025).
- Artigo da LACOR, 5 páginas, modelo oficial da MNR (arquivo do projeto).
- Documentação da Heltec WiFi LoRa 32 V3 (pinout e exemplos oficiais).
- Documentação do RadioLib (SX1262).
- Documentação do mbedTLS na versão do core ESP32 em uso, e do micro-ecc.
- Documentação do Python `cryptography` (ECDSA) e do FastAPI.
- RFC 6979 (ECDSA determinístico).

Fim do documento. Este arquivo deve ser atualizado a cada decisão que mude o contrato (pacote, serial, API) ou o escopo.